Transcript

Week 34: RestAPI

Video 1 - REST API

Have you ever wondered how different apps and systems talk to each other? REST APIs make that possible. They let software share data and functions across platforms, from mobile apps to cloud services. Let's see how REST APIs work and why they're essential in today's digital world.

An API acts as a bridge between different software applications, helping them communicate seamlessly.

APIs set rules for apps to communicate. To make this easier, think of an API as a waiter that takes your order and brings your food.

APIs work like waiters, delivering requests and responses. APIs play a crucial role in modern apps by enabling data exchange and service integration across platforms. There are several types: APIs include REST, SOAP, GraphQL, and WebSockets, each designed for specific needs. In this session, we'll concentrate on REST APIs and why they're vital in today's digital world.

REST, or Representational State Transfer, is a widely used architectural style for designing APIs. REST provides rules for building networked applications. This approach helps developers create scalable and maintainable web services.

One key principle of REST is its stateless model. Each request includes all required information. The server does not remember client state. This means the server never needs to recall previous interactions, making communication simple and reliable.

Another fundamental concept is that REST APIs are resource-focused. Uses resources identified by URLs. Here, data is organised with clear, standardised links, so information is easy to access and manage.

RESTful APIs are designed with several key characteristics to enhance scalability and efficiency. First, the client-server model separates the frontend and backend, making systems more modular and flexible. Statelessness means every client request includes all the required details, so the server never needs to remember previous interactions. Caching improves performance by allowing frequently accessed data to be stored and retrieved quickly. RESTful APIs support a layered system, using intermediaries like proxies and gateways to optimise communication. They also follow a uniform interface through standard HTTP methods such as GET, POST, PUT, and DELETE. Finally, servers can optionally enable code delivery, sending executable code like JavaScript to clients on demand.

RESTful APIs mainly use four HTTP methods to interact with resources. The first is GET, which retrieves data from the server. Think of this like viewing a webpage or downloading information. Next comes POST, which is used to create a new resource such as adding a new user. After that, PUT allows you to update details for an existing resource, for example, changing user information. Finally, DELETE is used to remove a resource like deleting a user profile.

Mastering these methods lets you work smoothly with RESTful APIs.

To illustrate how a RESTful API works, let's look at a simple example. Consider a GET request to /users/123, which asks the server for details about a specific user. The server then processes this and sends back relevant user information, such as their ID, name, and email in JSON format. This easy data access makes it straightforward for applications to read and use the data.

Why should we use RESTful APIs? There are several key reasons. First, they offer simple and easy use, which helps make them accessible to developers at all skill levels. Next, they're scalable, so applications can expand smoothly as their needs grow. RESTful APIs also have widespread adoption, especially in web and mobile apps, which means they're compatible with the latest technologies. And finally, they're cloud ready, seamlessly connecting with cloud-based services used in modern software.

Now it's your turn—apply what you've learnt about RESTful APIs by exploring real-world projects. Experiment, practise, and see how these skills can help you build smarter, more connected applications for the future. Give it a go and see where your tech journey takes you!

Video 2 - Architectural Style of REST

What do you think makes REST so widely used in modern applications? It's not just about sending data—it's about following a clear architectural style that ensures reliability, simplicity and scale. Understanding this structure is key to designing effective systems.

The six constraints of REST define how web services should be designed to ensure interoperability, scalability and performance. The first is the client-server model, which separates the user interface from backend logic. Next is stateless communication; every request has all the necessary information, because no session data is stored on the server. Cacheability allows responses to be cached, improving efficiency and performance. With a layered system, APIs can use multiple layers, like proxies or gateways, without affecting how the client and server communicate. A uniform interface is achieved by using standard methods and naming conventions, making APIs more consistent and easier to use. Finally, there's code on demand, where servers can send executable code—such as JavaScript—to the client, allowing added functionality when needed.

One of the most significant aspects of REST is the uniform interface, which simplifies interactions between clients and servers. The first element is resource identification, where each resource is uniquely identified using a URL. Next is standard formats, allowing clients to communicate and manipulate resources using formats like JSON or XML. Then comes self-descriptive messages, which means that requests and responses always carry all the information needed for processing. Finally, hypermedia as the engine of application state, or HATEOAS, helps clients discover actions by embedding available hyperlinks within responses.

RESTful APIs leverage standard HTTP methods to perform actions on resources. Starting with GET, you can retrieve resource data. Next, POST allows you to create new resources. Moving on, PUT is used to update existing records. With DELETE,

you can remove resources that are no longer needed. Finally, PATCH lets you partially update an existing resource.

RESTful API responses consist of three main components. First, the status code shows if a request has succeeded or failed, for example 200 OK for success, 404 Not Found for missing data, or 500 Internal Server Error for server issues. Next, headers give extra information about the response, such as content type and cache controls. Finally, the body provides the actual resource data, most often in JSON or XML format. A well-structured response makes it easier for clients to interpret API results efficiently.

RESTful APIs offer many advantages. First, scalability is achieved through statelessness, which enables load balancing and distributed systems. Next, flexibility means APIs can be consumed by different clients, including web, mobile and IoT. Moving on, interoperability is ensured by using standard HTTP methods and formats like JSON. Finally, performance improves when caching is used, as it speeds up response times and reduces server load.

When designing RESTful APIs, it's important to follow key best practices. Begin by using meaningful resource names in URLs to keep endpoints clear and descriptive. Next, always apply proper HTTP status codes, such as 200 OK for success, 404 Not Found for missing resources and 500 Internal Server Error for server failures. Then, ensure authentication and authorisation using methods like OAuth or JWT to protect sensitive data. For handling large datasets, make use of pagination and rate limiting to manage API load efficiently. Finally, enable versioning, for example with /api/v1/, to keep the API stable as it evolves over time. Applying these practices leads to well-structured and reliable RESTful APIs.

Put your REST knowledge into practice—try designing a simple RESTful API for a project of your own idea. Explore what you can build and see just how scalable and flexible your solutions can be!

Video 3 - Implementing RESTful APIs with Popular Frameworks

Imagine building the next viral app – would you choose speed or flexibility? Today, we're diving into RESTful APIs and comparing Flask with FastAPI to find out which toolkit could power your breakthrough project. Whether you're new to Python or aiming for scalable cloud solutions, let's unlock the secrets behind the APIs driving your favourite apps!

We start with what are RESTful APIs. REST stands for Representational State Transfer and is an architectural style for web services. On screen you can see that REST uses HTTP methods like GET, POST, PUT and DELETE.

Now let's look at the frameworks often used to build APIs. Why use Flask and FastAPI? Flask is simple, flexible and widely adopted. FastAPI gives you high performance and asynchronous capability, making it a modern solution for today's applications.

We're now Setting Up Flask. To install flask, type pip install flask in your terminal. Next, Import essentials with from flask import Flask, jsonify. Then, Create app by instantiating Flask.

Moving ahead, let's Define endpoint with the route /hello for GET requests. Inside this function, we'll Handle request and use jsonify to Send JSON response with a greeting.

Finally, to make your API live, Run app by checking the main module and then Start server with debug mode enabled.

Let’s learn how to Create CRUD APIs with Flask using a simple example. We begin by Importing Flask modules to get the right functionality. After that, we Initialise Flask app to start our project.

To manage our tasks, we Create list for tasks using a Python list. Next, we move to our API endpoints—first, we Define GET endpoint to retrieve all existing items and then Return all tasks as JSON to the requester.

For adding new tasks, we Define POST endpoint. With each POST request, we Add new task to list and then Confirm task added by sending back a simple message.

Finally, to test our API, we Run Flask in debug mode so you can safely make changes and see the results live.

Let’s get started with Setting Up FastAPI. First, install fast API by running pip install fastapi uvicorn. After installing, we Import FastAPI using a simple Python import.

Next, we Create app by instantiating FastAPI just like we did with Flask. Then, we Define GET endpoint for the /hello route and Handle request and respond by returning a welcome message in JSON format.

To make your API accessible, don’t forget to Run server with Uvicorn using the terminal command shown.

Let’s explore Creating CRUD APIs with FastAPI. We start by Importing FastAPI and Pydantic modules to enable data validation. Next, we Set up FastAPI app to initialise our API project.

For managing tasks, we Create list for tasks in Python. The Define Todo model step gives structure to our data using Pydantic for automatic validation.

Moving to API endpoints, we Create GET endpoint to fetch all tasks and Respond with all tasks to any client request.

To add new tasks, we Create POST endpoint. With every submission, we Add new task to the list and finally, Confirm addition by returning a message to the user.

Let’s compare Flask vs FastAPI across major features.

First is Performance. Flask offers lower performance, while FastAPI stands out with high performance due to async support.

Next, Ease of Use. Both frameworks are simple, but FastAPI is also modern and up-to-date.

Moving on to Async Support. Flask does not support asynchronous programming, whereas FastAPI provides built-in async capabilities.

When it comes to Data Validation, Flask requires manual checks, but FastAPI uses automatic validation through Pydantic models.

Finally, Documentation. Flask relies on manual documentation, but FastAPI generates documentation automatically with tools like Swagger and ReDoc.

Ready to build something brilliant? Put these frameworks to the test and create your own API project. Experiment, explore and see how far your ideas can go!

Video 4 - Rest API Implementation

What really happens behind the scenes when you tap your phone to book a ride or check your bank balance? At the heart of it all is a REST API making that connection work. Let’s explore how to implement one yourself—so you can start building smart, connected applications with confidence.

Let's walk through building a REST API in Python, step by step.

First, we import time to help us simulate API delays if we need to test response speed.

We bring in threading so our API can run more than one task at a time.

With requests, we can easily test our endpoints and make web calls from within Python.

Next, we import the main Flask features—these let us build an API, handle incoming requests and send out JSON responses.

For bigger projects, you can use FastAPI for faster performance and pydantic to check that your data is in the correct format.
Uvicorn helps run FastAPI apps.

Now, we create our Flask app. This line starts the API and connects everything together.

We initialise an empty list to store your todos. This list will hold every to-do sent to the API.

To allow users to view their todos, we create a GET route. This tells Flask to listen for requests asking for the to-do list.

When a GET request arrives, the function handles it and prepares the response.

Finally, we send the todos in JSON format. This makes our data easy to read for other applications or websites.

Each part of the code is essential to making your REST API work smoothly and reliably.

Let’s continue building our REST API.

First, we set up the create POST endpoint for adding new todos. This lets users send their own tasks to the server.

We then handle POST requests with a function that activates whenever someone submits a new todo.

Inside this function, we get JSON data from request—capturing the content sent by the user.

Next, we add new todo to list. This means every new entry is saved in our shared to-do list.

After storing the data, the API responds to the user. We confirm creation, send response with a success message and status code.

To finish, we need to make the app run. We set up the app runner function, which ensures everything is ready to launch.

Finally, we start server, threading ready. This runs the Flask application and sets it up for concurrent processing.

By adding and connecting each feature, you build a complete, practical API—step by step, each part working in harmony for reliable performance.

Let's explore building a REST API with FastAPI.

We start by creating the FastAPI app. This is the foundation for your API.

Next, we define a data model. By using a Pydantic model, we make sure each todo item has a title and a description, both as text. This keeps our data organised and ensures it is valid.

Moving on, we create a GET endpoint for todos. This allows users to request the list of all todos. When the endpoint is called, the function simply returns the todo list.

Now, we want users to add new todos. We create a POST endpoint for new todos. Here, the function adds the todo to the list after validation, guaranteeing only correct data is stored.

After a new todo is added, we confirm todo added with a friendly message.

Finally, we set up the app runner function, so you can use a command to start the server and make your API live.

By connecting each part, you create a reliable, well-validated REST API using FastAPI.

Now, let’s see how to test our APIs using threads and requests.

We begin by creating separate threads for the Flask and FastAPI servers, allowing both to run at the same time.

Afterwards, we start the Flask thread and start the FastAPI thread. This enables each API to work independently.

To make sure the servers are ready, we add a short pause using wait for servers to start.

Next, we test our endpoints. In the first block, we send a POST request to Flask to add a new todo.

After the call, we show the Flask response so we can verify the result. If there is any problem, we catch any Flask API error and print it out.

We then repeat the process for FastAPI.
Here, we send a POST request to FastAPI with different todo data and later show the FastAPI response. Any errors are handled by catching FastAPI API errors.

Finally, we wait for both threads to finish before ending our script.
This approach helps you experiment with both APIs at once, making integration testing smooth for your projects.

Now it's your turn—open your IDE and build your first REST API using the code we've covered today. Start with Flask, then try FastAPI to see which framework suits your style better.

Video 6 - Rest API Demo

Welcome to this demo session on Machine Learning Model Building and Deployment using REST API. This demonstration reflects a real-world MLOps workflow, showing how we go from model training all the way to a working API service. The goal here is to help you understand how data scientists and ML engineers collaborate to build modular and reproducible systems.

We will focus on four main areas, model training and evaluation, how we prepare, train and validate our model, model versioning and registry, how we organise and store models over time, REST API service for prediction, how to deploy and serve these models through a standard web API, and finally, automated testing and validation, ensuring that our system behaves reliably. The objective is simple, to build, evaluate and serve a machine learning model as a REST API using FastAPI. The technology stack is lightweight yet powerful, Python Scikit-Learn for model building, FastAPI for serving, JobLifter for serialisation, and PyTest for testing.

The end goal is to deliver a reproducible, modular and testable ML system that closely mirrors what's done in production, MLOps and environments. Let's look at what exactly we are building in this demo. We have broken it into two main phases, model training and deployment.

In phase one, model training handled by train.py, we start by loading the breast cancer dataset from Scikit-Learn. We then build a simple machine learning pipeline consisting of a standard scalar for normalisation and logistic regression classifier. The model is trained and evaluated on both training and testing spits to ensure generalisation.

Once trained, the model saves two types of outputs, the model artefacts, which goes into models directory, and a metrics file containing accuracy, precision, recall, F1 scores saved into metrics directory. In phase two, deployment, we switch to app.py. This is where we load the latest model from our registry and expose multiple REST API endpoints using FastAPI. For example, predict accepts input data, returns predictions, metrics, provides model performance metrics, retrain, allows background model retraining.

We'll also have health, checks whether the service and model are active or not. The FastAPI application also uses pedantic schemas for structured JSON responses and is tested using FastAPI's test client under PyTest. Essentially, this demo walks you through how a model transition from a local experiment to a deployable API with clear modular boundaries and automated testing.

Here's the architecture of our demo. It is designed to mirror how production ML services are structured, modular, maintainable, and clearly separated by responsibility. Everything starts with train.py. This is where we are loading the data set, splitting into train and test, then creating standard scalar logistic regression pipeline, evaluating the model and saving the model and metric.

Then the app.file will run. The file runs a FastAPI server that loads the data, sorry, loads the model. So if we load the model, it will expose rest endpoints that interact with the model for prediction, monitoring, and retraining.

To maintain input-output integrity, schema.py defines validation rules for all incoming and outgoing data. And finally, testAPI.py performs automated validation by calling the API endpoints directly, ensuring that the system is reliable before deployment. This flow ensures that data scientists, ML engineers, and QA teams can work in tandem without stepping on each other's toes.

Each layer handles its own scope cleanly. Here we break down the architecture into layers, mapping code files to their roles and how they connect. The training layer handles data loading, preprocessing, and model fitting.

That's your train.py. It connects to registry.py and app.py. Registry.py is our registry layer, which stores models and metrics. The API layer powered by app.py consumes the latest model from the registry and serves rest endpoints for real-time predictions. Then comes, so we have registry.py API that we spoke about.

Registry.py is connected to train.py and app.py API, serves prediction retrains connected to schema.py, registry.py. Then comes your validation implemented in schema.py. It enforces data integrity when we are predicting the output for the incoming data connected to app.py. And you have testing to see how model performs on the unseen data, which is again connected to app.py. So these are the layers that we have. Now, to tie everything together, here's a sequence of execution that happens in this demo. We begin by running our train.py, which will train the model, evaluate the model with metrics and saves artefacts to models and metrics.

Then we start the fast API service using uvicorn and then running the app.py on port 8000. This service automatically, sorry for that. So the service automatically loads the latest model and exposes the rest endpoints.

Then we have test endpoints. We then validate our endpoints using automated test via pytest. This confirms the predict and health routes return valid responses.

We also have retrain if required, we can retrain the model. And if required, you can also build a container using docker. These are the logical execution order summary.

Now let's move to our notebook and see how all this come together. So here. So we have this folder where we are doing ml API demo.

We have metrics, models, source, test and all. These are the folders where we are storing all the modules. Now when you go to source, we have these Python files.

If you see, let's start with train.py. A train.py purpose is very simple model training evaluation on both train and test using breast cancer data. So here, if you see what we are doing, we are inputting some libraries that is required for our model building and testing purpose. Then we are training the model and saving it.

So we are loading the data, picking up x and y component, feature names, and splitting the data into train and test, building our pipeline, which is for standard and logistic regression model, fitting the pipeline. And then we are versioning also. So we are saying version is equal to registry version stamp.

Now what it does in the registry file, registry.py, we have version stamp. So what it does is it simply picks up the time and create this percentage y, percentage mdb. So basically this will add as a suffix to a name that we will give.

So it will create a timestamp, which we can use to separate different versions of the model that we are training. So we pick up from here, version stamp. Then we get the model path, which is coming again from registry.modelPath for a particular version.

So this is the version. Now, if you look at registry.modelPath for version, it is nothing but the artefacts directory, which we have defined over here as models. So, artefacts directory is over here, see this, models.

And inside that we are storing the model with the version name. So what it will do, it will take today's date, time, at what time we have built, and then it will give extension as .joblib. So that's what it does. Then .joblib.dump, this will store the model, the feature names as well we want to store, and the version.

So all this will go to model path, which is inside this. I already have these files, see here. I have done multiple times.

And then it says registry.writeLatestSimLength. Now registry, if you go to latest sim length that we are writing here, it's basically we are picking up the destination, which is artefacts directory, and we create a latest .joblib model over there, which is nothing but the last model that you have trained is renamed to latest .joblib model. Why we do that? So that every time we retrain this model, the latest model will be picked up by our fast API server. So that's what we are doing over here.

This is our train.py. Then we are defining metrics for our model, and we pick up the predicted value, and we get the accuracy score, precision recall, and also F1 score.

The metrics, we are saying what's our version, what's our train score, what's our test, basically all the performance we are talking about, target names, feature names. And then the metric path is metrics path, which is basically this folder where we store these metrics in JSON format, which is again coming from registry.

And then we write the text here and there in the metrics folder, and then simply return the version of the model that we have trained and the model path. That's our train.py. So that's our train.py. Then after train.py, let's move to the next. Now understand one thing over here, we could have done these things in notebook as well.

But what we are doing, we are telling you how it works in real life. We put in a modular approach, train.py just simply trains everything and stores. Then we have schema.py. Now schema.py defines pedantic data models that describe what input data must look like when clients send a request and what output data must look like when the server sends a response.

The idea is to validate before applying model or even when we are sending the output. Now these schemas act as contracts between client and server. If the structure is wrong, fastAPI automatically rejects or corrects it before your model is even called.

Now this is connected to your app.py, train.py. And here first we are importing list and dictionary from typing and from pedantic we are importing base model and field. So list and dictionary will be used for type hints. Base model and field are from pedantic which fastAPI uses for data validation and documentation.

Base model will define schema classes, fields, lets you specify metadata, default value and field level validation. Then we are defining a class here, predict response, so predict request defines what kind of input JSON the predict endpoint expect. It expects one key instance which must contain a list of dictionaries and each dictionary represents one data record, for example one patient or a product.

Each dictionary maps feature names, their numerical value, so that's what it does. Then we have predict response, defines what output JSON your API will return after prediction. The fastAPI endpoint predict in app.py uses this model as its response type.

So that's your schema.py and what happens behind the scenes when someone calls predict in your API to get the prediction. The client will send a POST request with JSON input, fastAPI automatically parses and validates the input against predict request. If a required field is missing or has a wrong type, it will throw an error in return as unprocessable entity or 22.

Your endpoint in app.py receives Python object not raw JSON. Then you use that object to create a feature matrix for prediction. The model predicts probabilities and labels, fastAPI serialises your return Python dictionary into JSON that matches your predict response.

This means you never manually handle JSON parsing or validation, pedantic plus fastAPI do it automatically for you. So that's what it does. So the idea is that if I tell you if there is no schema.py, your API would accept any kind of JSON, even invalid or incomplete data.

It will fail silently or unpredictably during prediction, have no self-documentation in docs. By defining a schemas, fastAPI generates interactive API documentation. All request responses are typed, validated and documented.

So that's a benefit of it. Then registry.py, we already discussed, we have seen it, app.py is where the server is going to run. That's how we can think of it.

So you have app.py. Now what app.py is doing, it's basically implements. This file implements the fastAPI application that serves your trained machine learning model through a REST interface. So it allows you to load the latest trained model, predict on incoming JSON data, view metrics of the active model, check health version of the service and trigger retraining in the background.

In short, this file turns your static ML model into a live API service. Now here, what we are doing, we are importing the required libraries from fastAPI, fastAPI background task, JSON response from fastAPI responses, JobLib, JSON. From schema, we are picking a predict request, predict response that we went through.

We are also importing entire registry.py. From train, we are getting train and save. Now the app will run with the title ML REST API demo. Version is this.

That's the name we have given. And then we load latest. So this is basically loading the latest model from the model section.

So this will be the latest model. It will load that. If latest is none, then it will train the model and get the latest model.

So if there is no model available here, then it will train it. Then model payload is equal to load latest. Basically, the latest model is stored like this.

Then we have a health endpoint we are creating. And we are defining it like this. Status is OK.
Model version, model payload version. That's your health. App get metrics.

We are defining a metrics here. Version. Pick up the version of the model, the payload that we have here.

Then it says MP is equal to registry metrics path for version. So we have the metrics stored over here. It will pick up from there.

And if MP exists, it will load it into a text format. It will be a JSON only from the text. And then it will return a JSON response as code 404 content error metric not found.

If this is not there. If it is there, it will return this JSON content. Then we have retrain.

If I'm calling retrain, then it's a background task. It will train the model again. And it will store as a new version.

And then last, we have predict, which receives input. And then that input is your predict request. And then what it does, it loads the feature names.

It creates your data against the features like a dictionary. Get the probability from the model. Predict probability.

And it says if the probability is more than 0.5, then 1, else 0. And it returns the response. So that's what it does. So that's your app.part. So that's the entire structure we have here.

In addition to that, we have test API also. Here what we do is simply check how the services are running. It's working or not.

We check the client's health. This is the server health. If it is coming to 200, we have to check that.
And then it also has test predict, which predicts the model's performance on the test data. Or we can test for any other input as well. So that's your test API.

Now, whenever we create these stuff, like when we are deploying model, we also create readme.md file and requirements.txt. Requirements.txt will talk about all the libraries that you need to install before running it. And readme.md tells you step-by-step process that you need to follow. So see here.
So this is your ML API demo, real-world style. A minimal end-to-end example, features, quick start. Now, how do I run it? I need to first create a new environment and activate it.

Then install all the libraries that is mentioned in requirements.txt. After that, I need to run this, python-source-string.py. And then I need to run my server. Now, I already trained the model. If you see, I already have multiple models available that I have trained.

So I don't have to train it again. See here. I've trained so many times.

It's already there. I can simply run my server. For running it, I start my terminal. And I run my server. From readme.md, we got this uvicorn-source-app. And then port is 8000.
Let's run this. Now it's running here. See this? I can click on this link.

And here I can check the end point as health. And it says status is okay. And model version is this.
That's what it says over here. So my server is running. Because I already trained the model.
If I'm not trained, then I have to run train.py first. Okay. That's what we have to do.

Meaning that I need to start another terminal. And I have to say here python-m and src.train.py. Okay. That's what I need to do.

If I have not trained any model. So first this will run. It will train your first model, save it.

And then over here, this will run. But anyways, in the fast API itself, if you see, if you remember, we have mentioned that if it doesn't find a latest model, then it trains the model. So that's not a problem.

But in case you want to run separately, this is how you can run. Okay. So this is what we got here.
And then you will also have, you can also do the prediction if you want. You can send the data. You can predict the values.

That's what you can do. Okay. So all that is part of your deployment now.
So this is how you deploy a service like this. It will keep running. Okay.
See here. And you will also have, if you want, you can check here metrics. See here.
So these are the metrics. Version. Train.
What's the accuracy? What's the precision? Recall F1 on the test. How it is. Target names.
These are the feature names. So this is how we can deploy a model using REST API.
I hope you got a good idea about the model approach to your model deployment.
And this is generally how we build systems. Okay. And if I go to this, this, oh, 8000 and say docs, this is what I get.

See here. So this is just the detail about everything. Okay.
And here you can scroll to post predict. See here. Post predict.
Once you click on that, it comes like this. Post predict. See here.
Okay. And it also has the detail about everything. See this schema and all.
No responses at all. And here you can click on try it out. And when you get this, try it out section, it says instances.

And it says, I can put here details. I can say here. I need to provide all the details.
I can say here. And what was the feature names. So you can provide all of them.
So see here. All this you have to provide. So all this you have to provide.

I can simply copy it from here. And go here. And then you have to also provide values against it.
Some random value I'm providing. That's a data point. So you can do that.

And when you run it, which I'm just showing you when you execute. It will give you the values. See here.
Okay. So it says JSON invalid because I did not give the entire thing. Otherwise it will give you the response properly.

Because you have to paste so many things. So that way you can check as well. So that's how it works.
And this is how you can check everything if you want. And that's what the demo was all about. I hope you got a good idea about it.
Thank you.